

PHP

型検査・夢と理想と現実

PHP typing: the gap between ideal and reality.

補訂公開版

！ お前誰よ



- うさみけんた (@tadsan) / Zonu.EXE
- GitHub/Packagistでは id: zonuexe
- ピクシブ株式会社 pixiv事業本部
- Emacs Lisper, PHPer
 - Emacs PHP Modeのメンテナ引き継ぎました
 - 好きなリスプはEmacs Lispです
- Qiitaに記事を書いたり変なコメントしてるよ

pixiv



お絵描きがもっと楽しくなる場所を創る

pixiv 2018年度新卒
エントリー受付中

おしながき

おしながき

- 【はやわかり】PHPと型
- 型付けと型解析の考えかた
- 型と実装上の問題

最初から結論

PHPの
型付けは難しい

PHPのスカラー
型宣言は難しい

PHP7だからと言って、
やみくもにstringやint
などの型宣言を書くのは
おすすすめできない

思考停止での

`strict_types=1`

指定もおすすりめできない

コーディングには
PhpStormやPHPStan
を活用しましょう

静的解析で
CIしましょう

型宣言だけでなく
PHPDoc/PSR-5
の型も活用しよう

型の記法だけでは解決
できないことがあるの
で実装を工夫しよう

そんな話を
します

【はやわかり】

PHPと型

| PHPと型

- いわゆる「動的型」の言語
- PHP処理系は変数に型がない
- 変数にはどんな値も代入可能
- PHP7の型には二種類のモード(strict_types)がある

| PHPの型の種類

- スカラー型
- 複合型
- 特殊型
- ドキュメント上の擬似型

| スカラー型

- 論理型 (bool) true, false
- 整数型 (int) 1, 2, 3...
- 浮動小数点数 (float) 1.0,...
- 文字列型 (string) "abc"

| 複合型

- 配列型 (array)
- オブジェクト型 (object)
- 呼び出し可能型 (callable)
- 反復型 (iterable)

| 特殊型

- ノル型 (null)
- リソース型 (resource)
- 型ではないが返り値に書ける
 - void 返り値がないこと

| ドキュメント上の擬似型

- mixed 多様な型(複数)
- number int|false
- その他ツール固有の型表記

| declare(strict_types)

- ブロック(ファイル)単位でディレクティブを設定するための構文
- デフォルト(無指定時)は `strict_types=0`
- 引数と返り値についてキャストとエラーの振舞に影響(後述)

| declare(strict_types=0)

- 引数や返り値のスカラー型宣言と実際の値が一致しなかったとき、変換可能な値なら暗黙的に型変換
- オブジェクトが暗黙的にキャストされることはない

| declare(strict_types=1)

- 引数や返り値の宣言と実際の値が一致しなかったとき
TypeErrorが発生
- スカラー型・その他の型・
クラス/インターフェイスで
挙動に違いはない

| PHPのどこに型を書けるか

- 引数の型宣言
- 返り値の型宣言
- PHPDoc

| 引数の型宣言

- 関数/メソッドの仮引数リスト
- `function x(string $s, array $a)` のように書ける
- 書けるのは一部の型・擬似型とクラス・インターフェイス

！ 返り値の型宣言

- 仮引数リストの後に書ける
- `function y():int`のように書ける
- 基本は引数と同様
- 値を返さない場合に `: void` と書ける

| 型宣言のわかりにくい仕様

- 書ける型と書けない型がある
- self, array, callable, bool, float, int, string, iterable(7.1+), object(7.2+) だけが特別な意味を持つ
- それ以外はクラス/インターフェイス

| 型宣言に書けない型

- integer, boolean, double
などの型の別名は記述不可
- ちなみにstrはstringの別名…
ではないので覚えておいてく
ださい（型宣言に限らず）
- resourceは型宣言に書けない
- mixedやnumberも書けない

| PHPDocの型注釈記法

- クラスやメソッドの上にDoc Commentを書くことで型を記述できる
- あくまでコメントなのでコードの実行に影響を及ぼさず、IDEやツールの解析材料となる

| PHPDocの型注釈記法

- 型宣言では実現できない型が付けられる
- マジックメソッド/プロパティ
- ユニオン型 `Hoge|Fuga`
- 配列要素の型付け `int[]`

| PSR-5 (Draft)

- `array<string>`や
`ArrayObject<int,string>`と
書ける
- ……が、PhpStormは対応し
てない
- `ArrayObject|string[]`で代用

ここまでは
基本的な話

型付けと型解析の 考えかた

関数がどのような引数
を期待しどんな値を
返すのか明示したい

オブジェクトのメソッド
やプロパティが適切に
補完されると気持ち良い

プログラムが
間違ってることを
実行前に検証したい

ツールを
導入しよう

| 現行の高度な静的解析ツール

- PhpStorm (IDE)
- Phan
- PHPStan
- Psalm

Phanによる PHPコード静的解析

公開補訂版

ピクシブ株式会社
うさみけんた @tadsan



JAPAN phpcon 2016, 3rd November

PhanでPHPコード静的解析2018

Static analysis of PHP code using **Phan**.



2018-05-30 PHP勉強会@東京 #phpstudy

*php*ゆるふわCI入門

A casually guide for PHP Continuous Integration

pixiv

2019-03-30 PHPerKaigi 2019 Day 1
Nerima Coconeri Hall #phperkaigi

| PhpStorm

- よいところ
 - 導入が最も簡単
 - 十分な精度のエラー検出
- よくないところ
 - エラーの列挙ができないのでCIのフローに組み込めない

| Phan

- よいところ
 - 導入は比較的簡単
 - とても高い精度のエラー検出
- よくないところ
 - 大規模プロジェクトでは解析に時間がかかる

| Phanのすごいところ

- 連想配列にも型が付けられる
- array-shapes記法
- 自動で型推論もしてくれるが解析がすごすぎてレガシーで長大な関数から恐ろしい型が吐かれたりする

| PHPStan

- よいところ
 - 実行時情報を使用し高速
 - 高い精度のエラー検出
- よくないところ
 - Composer以外の依存関係があると導入にコツが要る

| Psalm

- よいところ
 - 独自の引数アサーション
 - 高い精度のエラー検出
- よくないところ
 - 大規模プロジェクトでは解析に時間がかかる

| つまりどれがいいのか (CI)

- 現時点ではそれぞれのツールで検出項目に一長一短があるため、使い分けるとよい
- PHPStanは高速なのでGitでブランチにpushするごとに検査するようなCIと相性がよい

| つまりどれがいいのか (エディタ)

- PhpStormは総合的に見て簡単に導入できて強力
- 自分で自由にハックしたいとか、どうしても予算が出ないなどの事情があればエディタとPHPStanがおすすめ

エディタ中立なPHP開発環境の現在

Modern editor-independent
development
environment for **PHP** 公開版

2018-11-23 Akiba Tokyo, Japan

VimConf 2018 [#vimconf](#)

プログラムが
間違ってることを
実行前に検証したい

“Phan is a static analyzer for PHP that prefers to minimize false-positives. Phan attempts to prove incorrectness rather than correctness.”

– <https://github.com/phan/phan>

型解析の 基本的な考え

これから説明する解析プロセスは便宜的に単純化したもので、実際のツールが報告する警告とは異なります

| 単純なコードで考える

`strlen(string $s): int`

`$length = strlen($a);`

`$length`は `int` が
代入される

`$a`には `string` が代入
されてるはず

単純なコードで考える

\$aは array が
代入される

\$lengthは int が
代入される

なんで
strlenに array を
渡してるの？

なんで
intと array を
足してるの？

\$a

\$length = strlen(\$a);

\$h = \$length + 1;

このコードに矛盾があるのは「実行しなくても」わかる

| 関数の型付けを考える

```
<?php
```

```
function x($a, $b) {  
    return $a . $b;  
}
```

string

string

string

\$aと\$bって
何を指定すればいいの？

結果の型は何？

文字列結合なので
両辺の値はstring

| 単純に解決できない関数

(number \$x, number \$b): number

(array \$x, array \$b): array

```
function x  
    return  
}
```

で、どっち？

PHPの + 演算子は
number + number
array + array
のどちらか

| PHPの型推論の限界

- 原理上コードの解析だけでは完全な静的検査は不可能
 - 変数やプロパティの動的性
 - 関数や演算子の多態性
 - インピーダンスミスマッチ

PHPの現実 と向き合う

| PHPDocで地道に型をつける

- 歴史あるコードは引数の仕様が形式化されたPHPDocで書かれてなかったり、崩れた形で書かれてたりする
- CIやIDEに磨かれてないと高確率で誤りを含んでる

| PHPDocに則ってない

```
//=====
// @params str $a
// @return true または false
//=====
function x($a, $b) {
```


| PHPDocに則るよう修正

```
/**  
 * @param string $a  
 * @return bool  
 */  
function x($a, $b) {
```

| 多態な関数設計を避ける

- 引数によって返り値の構造(型)が変化するような関数は避ける
- 例: `parse_url()`
 - 引数一つなら連想配列を返す
 - 第二引数に定数を渡すと文字列を返す

| mixed を避ける

- 引数や返り値が実装を見ないとわからなくなる
- json_encodeやvar_dumpみたいな多様なものを受け付ける関数以外はmixedと書くべき箇所は少ない

| arrayも極力控える

- 単にarrayとだけ書くと、配列の中身はmixed状態
- 「文字列が並ぶ配列」はstring[]
- 「決まったキーを持つ配列」の記法は標準化されていない

| @phan-param, @phan-return

- PhanのArray-shapes記法では
array{name:string,age:int}
のように記述できる

| 数値とインピーダンスミスマッチ

- 何かにつけて鬼門
- クエリパラメータやルーターの動的パラメータから数字列を取得する処理
- PDOから取得してきた数値

| クエリパラメータ

// 正の整数だけを期待

```
if (!is_numeric($_GET['user_id'])) {  
    throw new BadRequest;  
}
```

```
$user_id = (int)$_GET['user_id'];
```


| URL動的パラメータ

```
Route::get('user/{id}', function ($id) {  
    //  
})->where('id', '[0-9]+');
```

/user/000001 とか /user/000000000001
とか受け付けちゃって本当に大丈夫…?

| filter_var/filter_inputで安心？

```
filter_var(' 123', FILTER_VALIDATE_INT)  
// int(123)
```

| PHPの整数の範囲

- ビット数はアーキテクチャ依存の符号付き整数値
- 外部入力やDBのUNSIGNED BIGINTなカラムの値を(int)キャストして化ける

| スカラー型宣言とstrict_types

- キャストとつきあっていく覚悟が必要
- 通常のキャストとも別のルールで暗黙変換されることを認識しないと想定しない変換が走ることがある

| strict_typesの範囲

- declare(strict_types)はファイルごとに指定される
- 引数の型チェックは呼び出す側の指定が、返り値の型チェックは定義側の指定が反映される

レガシーコードとスカラー型

- 引数リストにむやみにstringやintと書かない
- PHPDocならキャストは起こらないので動作に影響はしない

静的解析結果と
どうやって向き
合っていか

| 既存プロジェクトを検査すると

- 数千行にも及ぶおびただしい警告が出力されることもざら
- しかしそれよりも、そのコードが「たしかに動いている」事実が重要

| 大事ななのは傷口を広げないこと

- 警告が出てるから間違ったコードではない
- 警告の量が減ればメンテしやすい状況に近付くのだと捉える

| 差分を重要視する

- リファクタリングの前後で Phanの出力ログのdiffをとり、行が増えなければ、間違った書き換えはしていない可能性は高い
- ロジックの正しさは保障できないのでテストなどは別途必要